

1. Project Idea

The project aims to design and implement an intelligent system to solve the N-Queens Problem, which is a classical problem in computer science and artificial intelligence.

The objective is to place N queens on an $N \times N$ chessboard such that no two queens attack each other. This means:

- ♠ No two queens share the same row
- ♠ No two queens share the same column
- ♠ No two queens share the same diagonal

To efficiently represent the board, a one-dimensional array is used instead of a two-dimensional matrix. Each index represents a row, and the value at that index represents the column position of the queen.

Example:

For $N = 4$

Board = [1, 3, 0, 2]

This means:

- ♠ Row 0 $\rightarrow$ Column 1
- ♠ Row 1 $\rightarrow$ Column 3
- ♠ Row 2 $\rightarrow$ Column 0
- ♠ Row 3 $\rightarrow$ Column 2

The system allows the user to select a specific algorithm and input the value of N, enabling focused experimentation and analysis of different approaches.

2. Main Functionalities

The system provides the following functionalities:

- ♠ Accept user input for number of queens (N)
- ♠ Allow the user to select the desired algorithm:
 - Backtracking (DFS)
 - Best-First Search
 - Hill Climbing with Random Restart
 - Genetic Algorithm

- ♠ Solve the problem using the selected algorithm
- ♠ Validate queen placements using constraints
- ♠ Display the solution in:
 - Array representation
 - Visual board format (ASCII)
- ♠ Track performance metrics such as:
 - Number of steps
 - Execution time
 - Conflict history (for heuristic algorithms)
- ♠ Allow users to manually compare different algorithms by running them individually

3. Similar Applications in the Market

Although the N-Queens problem is primarily academic, similar concepts are widely used in real-world applications such as:

- ♠ Constraint Satisfaction Problem (CSP) solvers
- ♠ AI-based scheduling systems
- ♠ Puzzle-solving applications
- ♠ Optimization and resource allocation systems

Examples include:

- ♠ AI solvers and open-source implementations on platforms like GitHub
- ♠ Constraint programming frameworks
- ♠ Optimization tools such as Google OR-Tools

However, most available applications focus on a single solving technique. This project distinguishes itself by integrating multiple algorithms and enabling performance comparison between them.

4. Literature Review (Related Papers)

Paper 1 “Malak”: NSCR Algorithm for the N-Queens Problem

The article presents the Non-Sequential Conflict Resolution (NSCR) algorithm, a hybrid method for solving the N-Queens problem by combining backtracking with iterative conflict repair.

Unlike traditional algorithms that immediately backtrack when conflicts occur, NSCR attempts

to relocate conflicting queens to positions with the fewest threats before backtracking. This approach reduces unnecessary computations, lowers memory usage, and improves scalability. Experimental results show that NSCR outperforms traditional methods such as BFC, Lookahead, and CSP in both execution time and memory efficiency, especially for large board sizes up to 1000 queens.

Conclusion

The paper concludes that NSCR is a more adaptive and scalable solution for the N-Queens problem compared to traditional approaches. Its hybrid strategy successfully balances speed and memory efficiency, making it suitable for large-scale combinatorial optimization tasks. Although the algorithm may still encounter issues in certain edge cases and retains a high worst-case complexity, it significantly reduces runtime and redundant computations in practice.

Relevance to Our AI N-Queen Project

This article is highly relevant to our AI project because it demonstrates how intelligent conflict resolution techniques can improve the efficiency of solving the N-Queens problem. The NSCR approach aligns with AI concepts such as heuristic search, optimization, and adaptive decision-making. By integrating conflict minimization instead of relying solely on brute-force backtracking, the algorithm provides ideas for building a faster, more scalable, and memory-efficient AI-based N-Queens solver, especially for handling large board sizes.

Paper 2 “Hana”: N-Queen Problem (Backtracking Algorithm)

The article explains the N-Queen problem and how the backtracking algorithm is used to solve it efficiently. The method places queens row by row while checking for conflicts in columns and diagonals. If a conflict occurs, the algorithm removes the last queen placement and tries another position. This process continues until a valid arrangement is found or all possibilities are exhausted. Backtracking reduces unnecessary computations by pruning invalid configurations early.

Conclusion

The backtracking approach is an effective and widely used method for solving the N-Queen problem because it systematically explores solutions while avoiding invalid paths. Even though its time complexity is approximately $O(N!)$, it performs much better than brute-force methods by eliminating impossible configurations early in the search process.

Relevance to Our AI N-Queen Project

This article is directly relevant to our AI project because backtracking is one of the fundamental search and problem-solving techniques in artificial intelligence. It demonstrates concepts such as state-space search, constraint satisfaction, and intelligent pruning. Understanding backtracking provides a strong foundation for developing more advanced AI-based N-Queen solvers and for

comparing traditional approaches with newer optimization techniques like heuristic search or conflict-resolution algorithms.

Paper 3 “Moahnnad”: Survey: Genetic Algorithm for N-Queens (2025)

Classification: Identifies N-Queens as an NP-hard problem.

Mechanism: Evaluates how Genetic Algorithms (GA) use natural selection (Crossover/Mutation) to navigate massive search spaces.

Focus: Compares "Permutation Encoding" techniques to eliminate row/column conflicts by design.

Conclusion

Efficiency: GA is superior to Brute-Force for large N . **Critical Factors:** Success depends on a precise Fitness Function (conflict counter) and maintaining Diversity to avoid getting stuck in Local Optima.

Relevance

Justification: Provides the theoretical "Why" for using Heuristic search in your project.

Benchmarking: Serves as a reference point to compare your Best-First and Hill-Climbing results against Evolutionary methods.

Modernity: Citing a 2025 paper adds high academic value to your Literature Review.

Paper 4 “Mohammed”: An Almost Perfect Heuristic for the N Nonattacking Queens Problem (1990)

This paper introduces a heuristic-based approach for solving the N-Queens problem with minimal or no backtracking. The proposed method improves search efficiency by selecting promising positions for queens instead of exploring all possible states randomly. The study demonstrates that heuristic techniques can significantly reduce computation time and search complexity compared to traditional exhaustive methods.

Conclusion

The research concluded that heuristic search methods are highly effective in solving the N-Queens problem efficiently. By guiding the search process intelligently, the algorithm reduces unnecessary exploration and improves overall performance.

Relevance to Our Project

This paper is highly relevant because it supports the use of heuristic and Best-First Search concepts in our project. It also explains how intelligent search strategies can improve performance compared to traditional search algorithms.

Paper 5 “Omar”: N-Queens Problem Optimization Using Various Memetic Algorithms

Khanna, Bhardwaj, and Chopra (2017) addressed the well-known limitations of the standalone Genetic Algorithm (GA) in solving the N-Queens Problem (NQP), which is classified as an NP-hard combinatorial problem with a solution space complexity of $O(n!)$. The authors proposed the use of Memetic Algorithms (MA) — a hybrid approach that combines the global search capabilities of GA with the local exploitation power of local search techniques — as a more effective alternative. Two MA variants were evaluated: the Hill Climbing Local Search-based Memetic Algorithm (HCLSMA) and the Simulated Annealing Local Search-based Memetic Algorithm (SALSMA). The proposed MA solves the NQP in two distinct phases: the first phase focuses on satisfying hard constraints by ensuring no two queens share the same row, column, or diagonal, while the second phase refines the resulting feasible solutions by minimizing any remaining soft constraint violations. Experimental results revealed that HCLSMA consistently outperformed both SALSMA and the standalone GA across all tested board sizes ranging from $N=8$ to $N=40$, achieving a 100% convergence rate in all cases regardless of the number of iterations. In contrast, the standalone GA failed to produce any valid solutions for board sizes of $N=24$ and beyond, highlighting its scalability limitations. In terms of time complexity, HCLSMA also demonstrated lower execution times compared to both SALSMA and GA. These findings strongly support the conclusion that hybridizing Hill Climbing with an Evolutionary Algorithm significantly improves solution quality, convergence reliability, and computational efficiency when solving large-scale instances of the N-Queens Problem.

Reference (IEEE format):

V. Khanna, A. Bhardwaj, and S. Chopra, "N-Queens Problem Optimization Using Various Memetic Algorithms," **International Journal of Engineering Development and Research**, vol. 5, no. 2, pp. 1206–1214, 2017.

Paper 6 “Sayed”: Bandit-Based Random Mutation Hill-Climbing

Liu, Perez-Liebana, and Lucas (2016) proposed a novel enhancement to the traditional Random Mutation Hill-Climbing (RMHC) algorithm by incorporating a multi-armed bandit-based selection mechanism, resulting in what they termed the Bandit-Based Random Mutation Hill-

Climbing (Bandit-RMHC) algorithm. The standard RMHC operates by randomly selecting a neighbouring solution and accepting it only if it performs equally well or better than the current best solution — a straightforward but often inefficient approach. The key limitation of this method is that it treats all possible mutations equally, with no intelligent prioritization of which gene or variable to mutate next. To address this, the authors modeled each gene in the solution as an independent multi-armed bandit unit, where an urgency score — derived from a modified Upper Confidence Bound (UCB) formula — determines which gene should be mutated at each iteration. This mechanism effectively balances the trade-off between exploration of untested mutations and exploitation of known beneficial changes. The algorithm was evaluated on two standard benchmark problems: the OneMax problem and the Royal Road function, both in noise-free and noisy environments. Experimental results demonstrated that Bandit-RMHC significantly outperformed standard RMHC across all tested scenarios, requiring up to ten times fewer fitness evaluations in noisy conditions. Additionally, the number of evaluations needed by Bandit-RMHC scaled approximately linearly with problem dimension, highlighting its computational efficiency for large-scale discrete optimization problems. These findings suggest that integrating intelligent selection strategies into Hill-Climbing algorithms can substantially improve both convergence speed and solution quality, making Bandit-RMHC a promising approach for complex optimization problems where fitness evaluations are computationally expensive.

Reference (IEEE format):

J. Liu, D. Perez-Liebana, and S. M. Lucas, "Bandit-Based Random Mutation Hill-Climbing," *arXiv preprint arXiv:1606.06041*, 2016.

Paper 7 “Hager”: Genetic Algorithm vs Traditional Search Techniques (2023)

This paper compares the performance of the Genetic Algorithm with traditional search methods such as BFS and DFS in solving the N-Queens problem. The researchers found that Genetic Algorithms work faster for large board sizes because they reduce the search space and reach solutions more efficiently. In contrast, traditional search methods become slower as the number of queens increases due to the huge number of possible states.

Conclusion

The study concluded that Genetic Algorithms are more suitable for solving complex N-Queens problems and provide better performance for optimization tasks.

Relevance to Our Project

This paper supports the comparison between Genetic Algorithm and classical search techniques used in our project.

Paper 8 “Nada”: Solving N-Queens Using Subproblem-Based Genetic Algorithm (2018)

This paper presents an improved Genetic Algorithm approach by dividing the N-Queens problem into smaller subproblems. This technique reduces computational complexity and improves execution speed. The study showed that the modified Genetic Algorithm achieves faster convergence and produces better results than the standard GA approach.

Conclusion

The research proved that dividing the problem into smaller parts can improve the efficiency and performance of Genetic Algorithms.

Relevance to Our Project

This paper provides useful optimization ideas that can help improve the Genetic Algorithm implementation in our project.

Paper 9 “Rahma”: N-Queens using Genetic Algorithm: A Comparative Study (2025)

This paper compares the performance of the Genetic Algorithm and Backtracking in solving the N-Queens problem. The study showed that Backtracking guarantees accurate solutions but becomes slow and inefficient as the board size increases. On the other hand, the Genetic Algorithm reduces computation time and handles large values of N more efficiently by using optimization techniques such as selection, crossover, and mutation.

Conclusion

The research concluded that Genetic Algorithms are more scalable and faster for large N values, while Backtracking is more suitable for smaller problems where exact solutions are required.

Relevance to Our Project

This paper is highly relevant because it directly compares two algorithms used in our project: Genetic Algorithm and Backtracking. It also helps justify the advantages and limitations of each approach.

Paper 10 “Basmala”: Local Search Analysis N-Queens Problem (2018)

This paper studies local search techniques for solving the N-Queens problem, especially Hill Climbing. The algorithm works by gradually improving the current solution to reduce conflicts

between queens. The study showed that Hill Climbing is fast and efficient, but it may stop at local optima before finding the best solution.

Conclusion

The research concluded that Hill Climbing performs well for quickly finding solutions, but additional techniques are needed to avoid getting stuck in local optima.

Relevance to Our Project

This paper supports the use of the Hill Climbing algorithm in our project and explains its strengths and weaknesses in solving the N-Queens problem.

Paper 11 “Amr”: Divide and Conquer under Global Constraints: A Solution to the N-Queens Problem

The paper introduces a Divide and Conquer strategy as an alternative approach to solving the N-Queens problem.

Instead of exploring the board step-by-step like traditional backtracking, the algorithm divides the chessboard into smaller independent subproblems, solves them separately, then combines the partial solutions while maintaining the global constraints of the problem.

The main idea is:

Divide:

Split the $N \times N$ board into smaller sections or sub-boards.

Conquer:

Solve each smaller section independently by placing queens without conflicts.

Combine:

Merge the partial solutions into one complete valid board configuration while ensuring:

- ♠ no queens share the same row,
- ♠ no queens share the same column,
- ♠ no queens attack diagonally.

The paper explains that this method reduces the search complexity compared to exhaustive recursive backtracking because:

it decreases the branching factor, allows partial parallel solving, and avoids repeatedly checking the entire board.

The study also compares divide-and-conquer with classical backtracking and shows that:

backtracking is accurate but slower for large N, divide-and-conquer improves scalability and execution time for bigger boards.

The paper concludes that divide-and-conquer can be an efficient alternative for large-scale N-Queens problems, especially when combined with optimization techniques.

Relation Between the Paper and Your Project

This paper is related to your project mainly as a comparison approach against your implemented algorithms.

1. Relation to Your Backtracking Solution

Your DFS/backtracking implementation:

Places queens row by row, recursively explores all valid states, and backtracks when conflicts occur.

Example from your code:

```
if isvalid(i,j):
```

```
    board[i] = j
```

```
    dfs(i+1, ...)
```

Paper Contrast

The divide-and-conquer paper proposes avoiding deep recursive exhaustive exploration by:

splitting the problem first, solving smaller chunks independently, then combining results.

So the paper provides a theoretical contrast to your classical recursive approach.

5. Dataset Used

This project does not rely on a traditional dataset, as the N-Queens problem is a generated problem rather than a data-driven one.

Instead, the input is dynamically provided by the user in the form of the board size (N). Each input represents a unique test case.

For experimental evaluation, multiple values of N (e.g., from small to large sizes) are used to analyze and compare the performance of different algorithms.

6. Details of Algorithms and Experimental Results

6.0 Heuristic Functions Design

To guide the search algorithms efficiently, two heuristic functions are defined to evaluate the quality of a given state.

Heuristic Function 1 (h1): Number of Attacking Pairs

Definition

- ♠ This heuristic calculates the total number of pairs of queens that are attacking each other on the board.

Explanation

- ♠ A pair of queens is considered in conflict if they:
 - ♣ Share the same column
 - ♣ Lie on the same diagonal

The heuristic value is computed by counting all such attacking pairs.

Formula

$h1$ = total number of conflicting queen pairs

Properties

- ♠ $h1 = 0 \rightarrow$ Goal state (valid solution)
- ♠ Lower values indicate better states
- ♠ Widely used in Best-First Search and Hill Climbing

Heuristic Function 2 (h2): Number of Conflicted Queens

Definition

This heuristic counts the number of queens that are involved in at least one conflict.

Explanation

Instead of counting pairs, this function counts how many queens are تحت threat (in conflict with any other queen).

Formula

$h2$ = number of queens that have at least one conflict

Properties

- ♠ $h_2 = 0 \rightarrow$ Goal state
- ♠ Provides a different evaluation perspective than h_1
- ♠ Helps diversify search behavior

Usage in Algorithms

- ♠ Best-First Search:
Uses h_1 to expand the state with the minimum number of conflicts.
- ♠ Hill Climbing:
Uses h_1 or h_2 to move toward a better neighboring state.
- ♠ Genetic Algorithm:
Fitness function can be defined as:
$$\text{Fitness} = 1 / (1 + h_1)$$

(to convert minimization into maximization)
- ♠ Using multiple heuristic functions improves the robustness of the system by allowing different evaluation strategies and reducing the chances of getting stuck in poor local optima.
- ♠ Two heuristic functions were designed (h_1 and h_2), but h_1 was primarily used in the implementation due to its higher precision.

6.1 Backtracking (DFS)

The Backtracking algorithm is a systematic search approach that explores all possible configurations.

This algorithm does not use heuristic functions; it relies on systematic search.

Key idea:

- ♠ Place queens row by row
- ♠ Check validity before placing
- ♠ Backtrack when a conflict occurs

Characteristics:

- ♠ Guarantees finding a solution
- ♠ Time complexity is high ($O(N!)$)
- ♠ Suitable for smaller values of N

State

- ♠ The state is represented as a partial solution using a one-dimensional array of size N , where each index represents a row and the value represents the column position of the

queen. At any step, the state contains the positions of queens placed in the first k rows.

Action

- ♠ The action is placing a queen in the next row in a column that does not violate any constraints (no same column or diagonal conflicts with previously placed queens).

State Space

- ♠ The state space consists of all possible configurations of placing queens on the board. In the worst case, the size of the state space is $O(N!)$, but it is significantly reduced using pruning (backtracking) by eliminating invalid states early

6.2 Best-First Search

Best-First Search is a heuristic-based algorithm that selects the most promising state based on an evaluation function.

This algorithm uses heuristic function $h1$ (number of attacking pairs) to evaluate and select the most promising state.

Key idea:

- ♠ Use a heuristic (number of conflicts)
- ♠ Expand the state with the least conflicts first

Characteristics:

- ♠ Faster than brute-force approaches
- ♠ Performance depends on heuristic quality
- ♠ Does not always guarantee optimal solution

State

- ♠ A state is represented as a configuration of queens on the board using a one-dimensional array. It may represent a complete or partial assignment of queens.

Action

- ♠ The action is generating successor states by placing a queen in a valid position in the next row or modifying positions to reduce conflicts. Each successor is evaluated using a heuristic function.

State Space

- ♠ The state space includes all possible board configurations. The algorithm explores the most promising states first based on the heuristic (e.g., number of conflicts), reducing the need to explore the entire space.

6.3 Hill Climbing (Local Search)

Hill Climbing is an optimization algorithm that improves the current state iteratively.

The heuristic function $h1$ is used to evaluate neighboring states and guide the search toward fewer conflicts.

Key idea:

- ♠ Start with a random configuration
- ♠ Move to a neighboring state with fewer conflicts
- ♠ Restart if stuck in a local optimum

Characteristics:

- ♠ Very fast
- ♠ May fail without random restarts
- ♠ Suitable for large N .

State

- ♠ A state is a complete configuration of N queens on the board, represented by a one-dimensional array where each row has exactly one queen.

Action

- ♠ The action involves moving a queen within its row to another column to reduce the number of conflicts. The algorithm selects the neighboring state with the lowest number of conflicts.

State Space

- ♠ The state space consists of all possible complete board configurations. The algorithm searches locally within neighboring states and may get stuck in local optima, which is why random restart is used.

6.4 Genetic Algorithm

The Genetic Algorithm is inspired by natural selection.

The fitness function is derived from heuristic $h1$, where fewer conflicts result in higher fitness.

Steps:

- ♠ Initialize a random population
- ♠ Evaluate fitness (based on conflicts)
- ♠ Apply selection, crossover, and mutation
- ♠ Repeat for several generations

Characteristics:

- ♠ Works well for large search spaces
- ♠ Probabilistic approach
- ♠ Balances exploration and exploitation

State

- ♠ A state (individual) is represented as a chromosome, which is a one-dimensional array of size N . Each gene represents the column position of a queen in a specific row.

Action

The actions include genetic operations:

- ♠ Selection: choosing the best individuals based on fitness
- ♠ Crossover: combining two parent solutions to produce offspring
- ♠ Mutation: randomly modifying a gene to maintain diversity

State Space

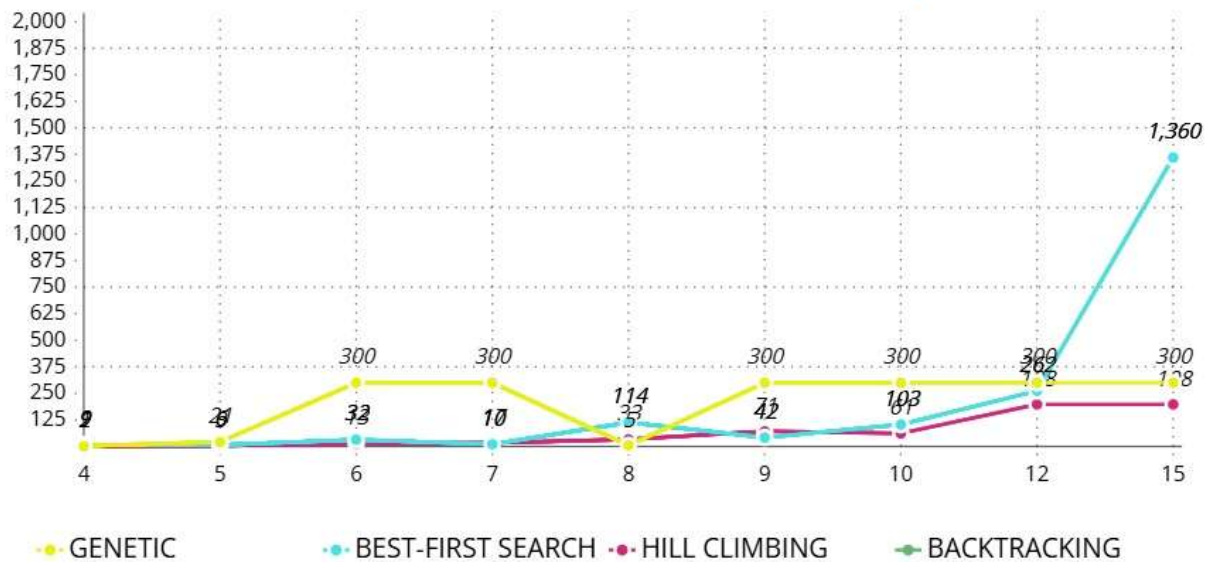
- ♠ The state space includes all possible chromosomes (solutions). The algorithm explores this space probabilistically by evolving a population of candidate solutions over multiple generations.

6.5 Experimental Results

Performance Comparison

Algorithm	Speed	Accuracy	Scalability
Backtracking (DFS)	Slow	High	Low
Best-First Search	Medium	Medium	Medium
Hill Climbing	Fast	Medium	High
Genetic Algorithm	Medium	Medium	High

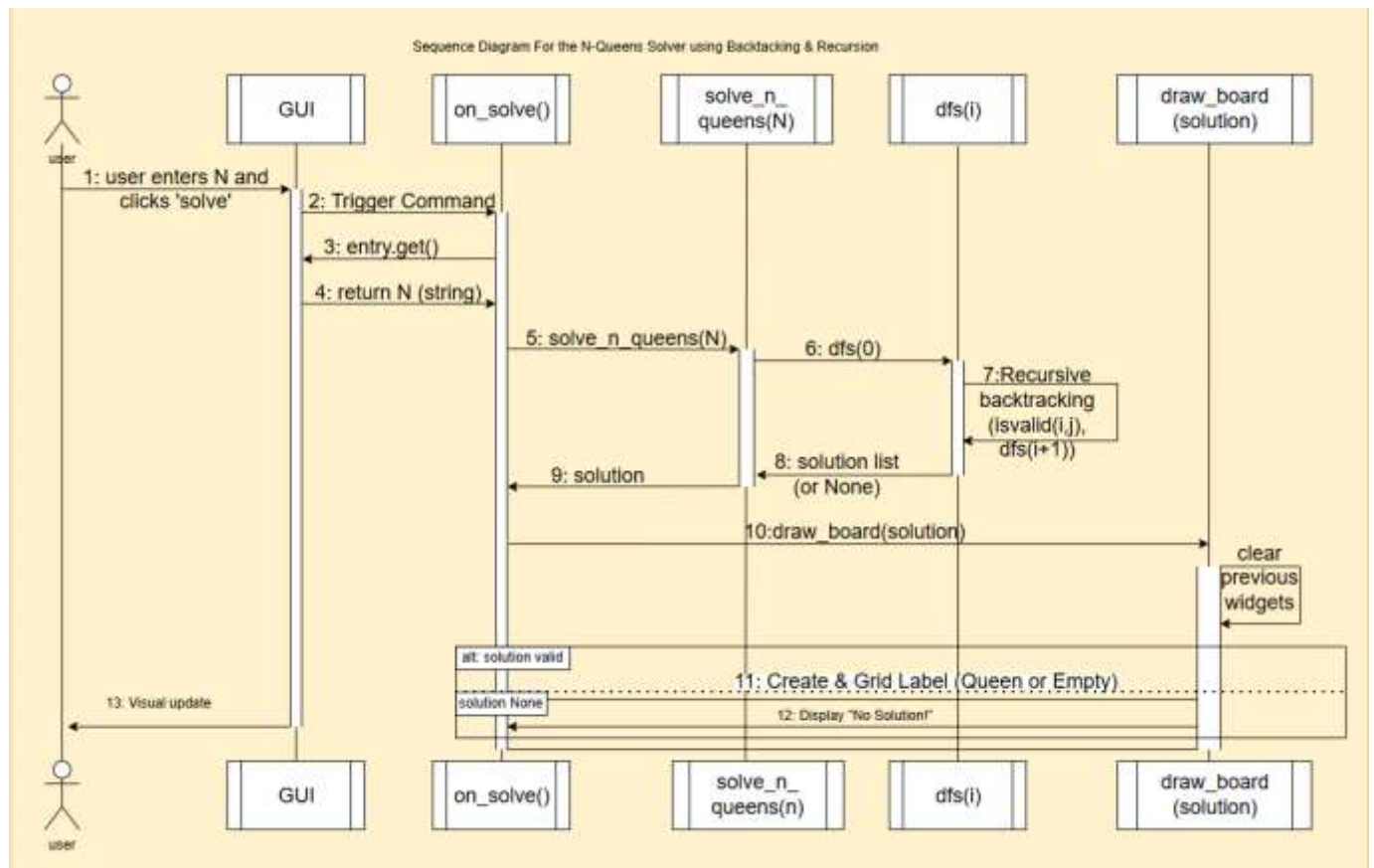
Number of N. vs Number of Steps



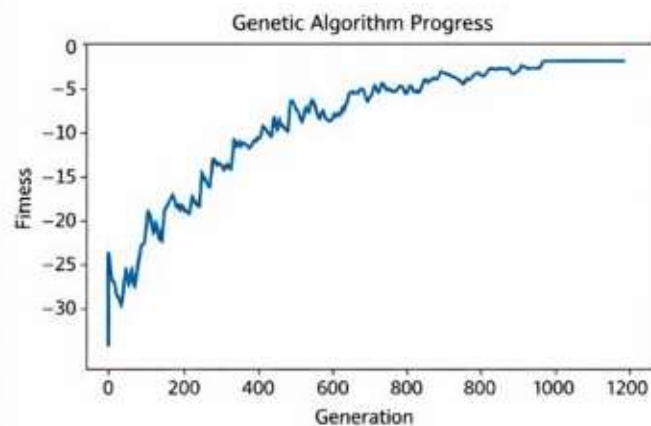
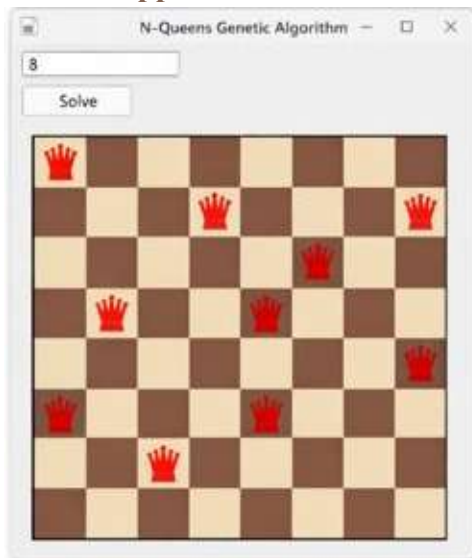
Created by charts.livegap.com

6.6 Graphs and Visual Results

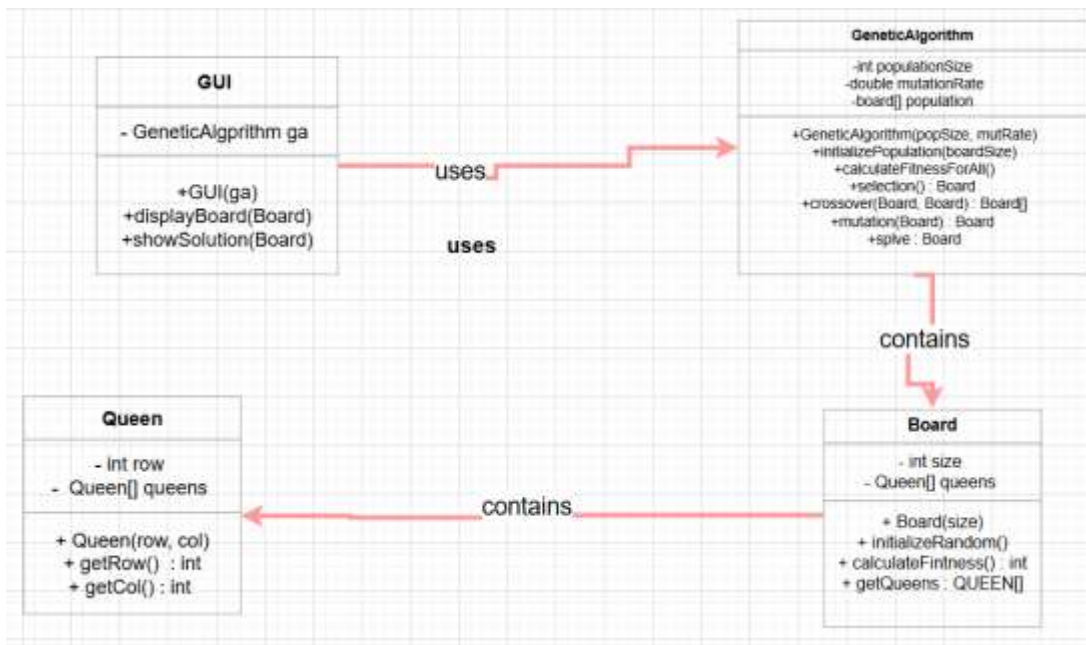
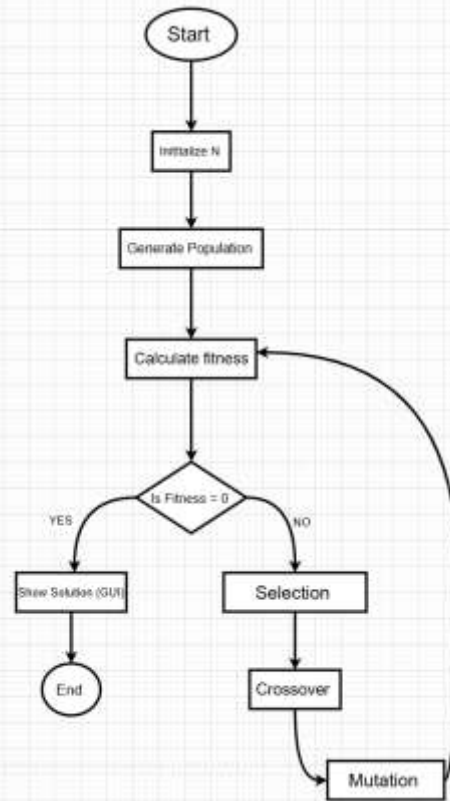
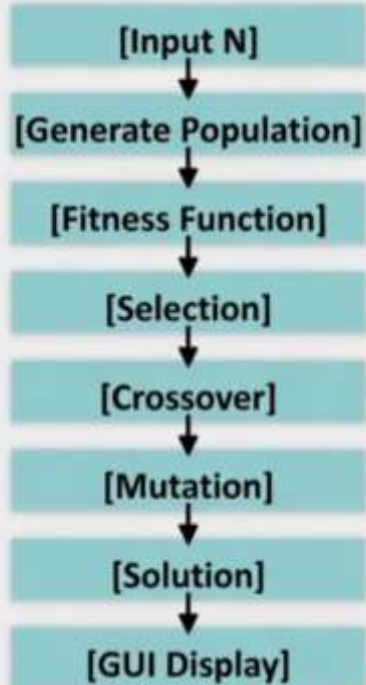
BackTracking:



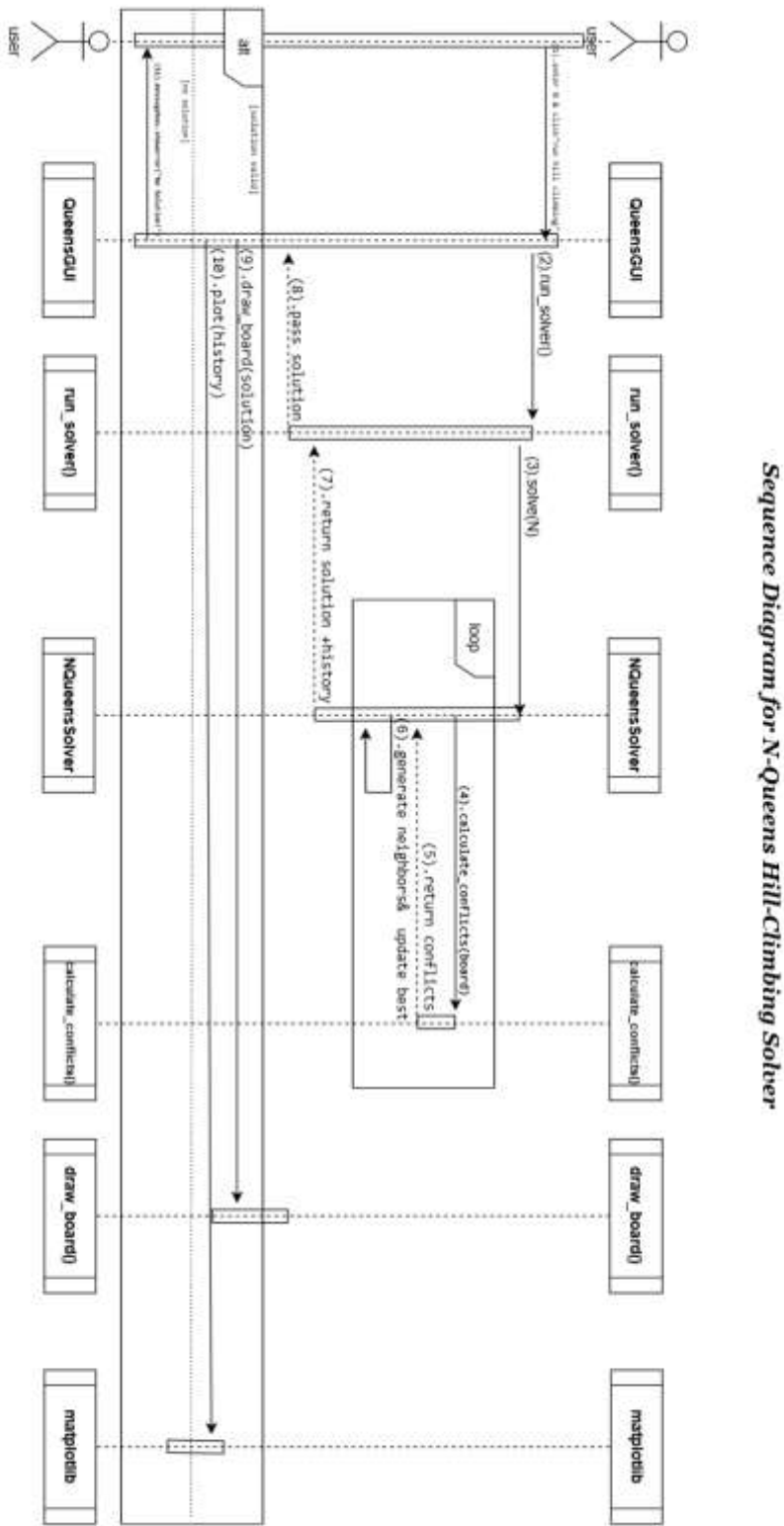
Genetic Approach:



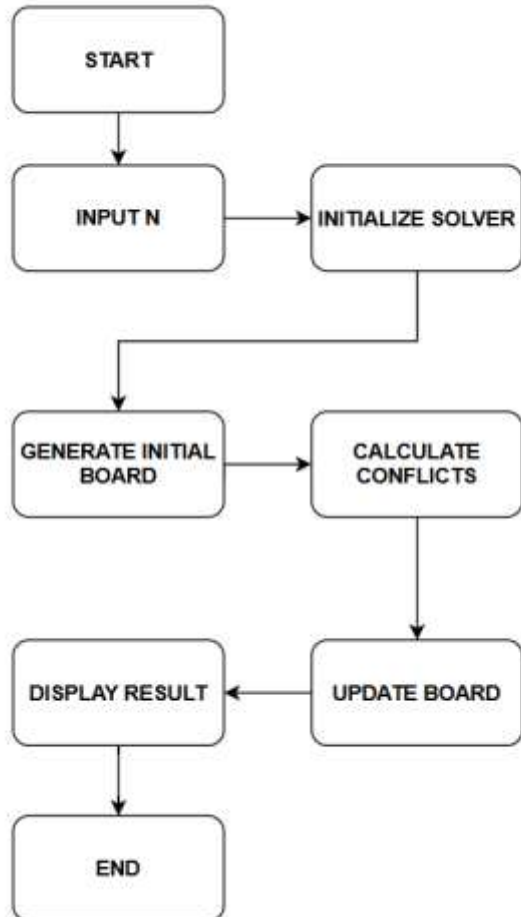
Genetic Algorithm Block Diagram with GUI



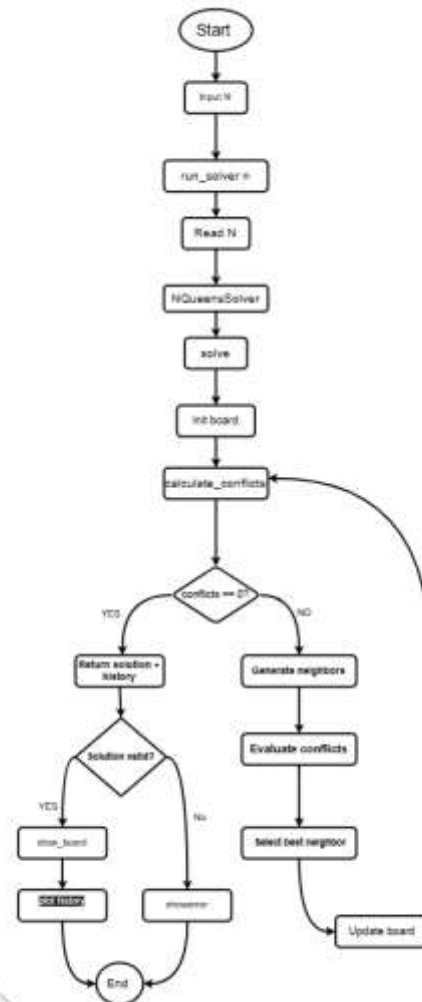
Hill-Climbing:



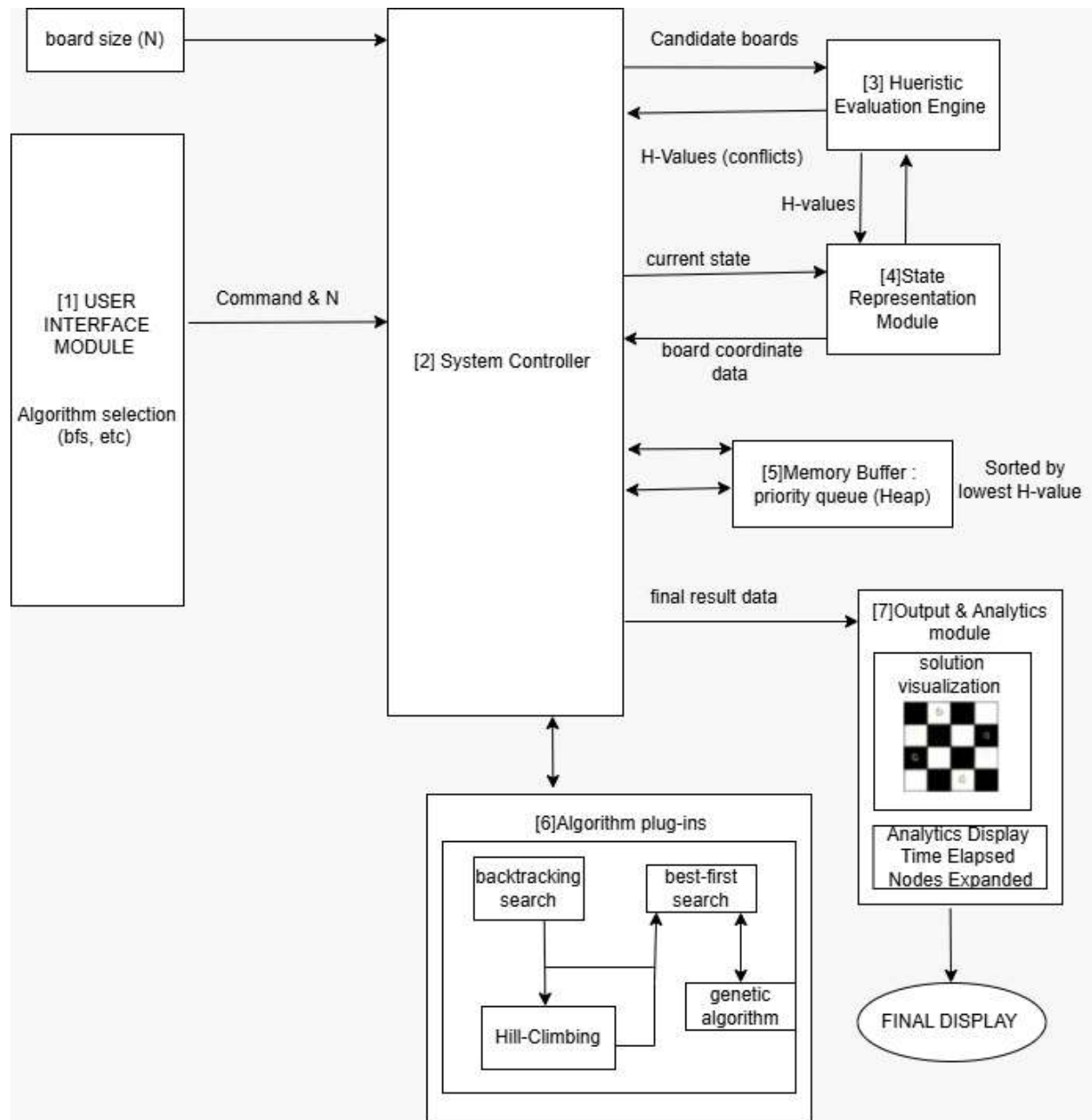
**BLOCK DIAGRAM FOR N-QUEENS SOLVER
BY HILL-CLIMBING ALGORITHM**

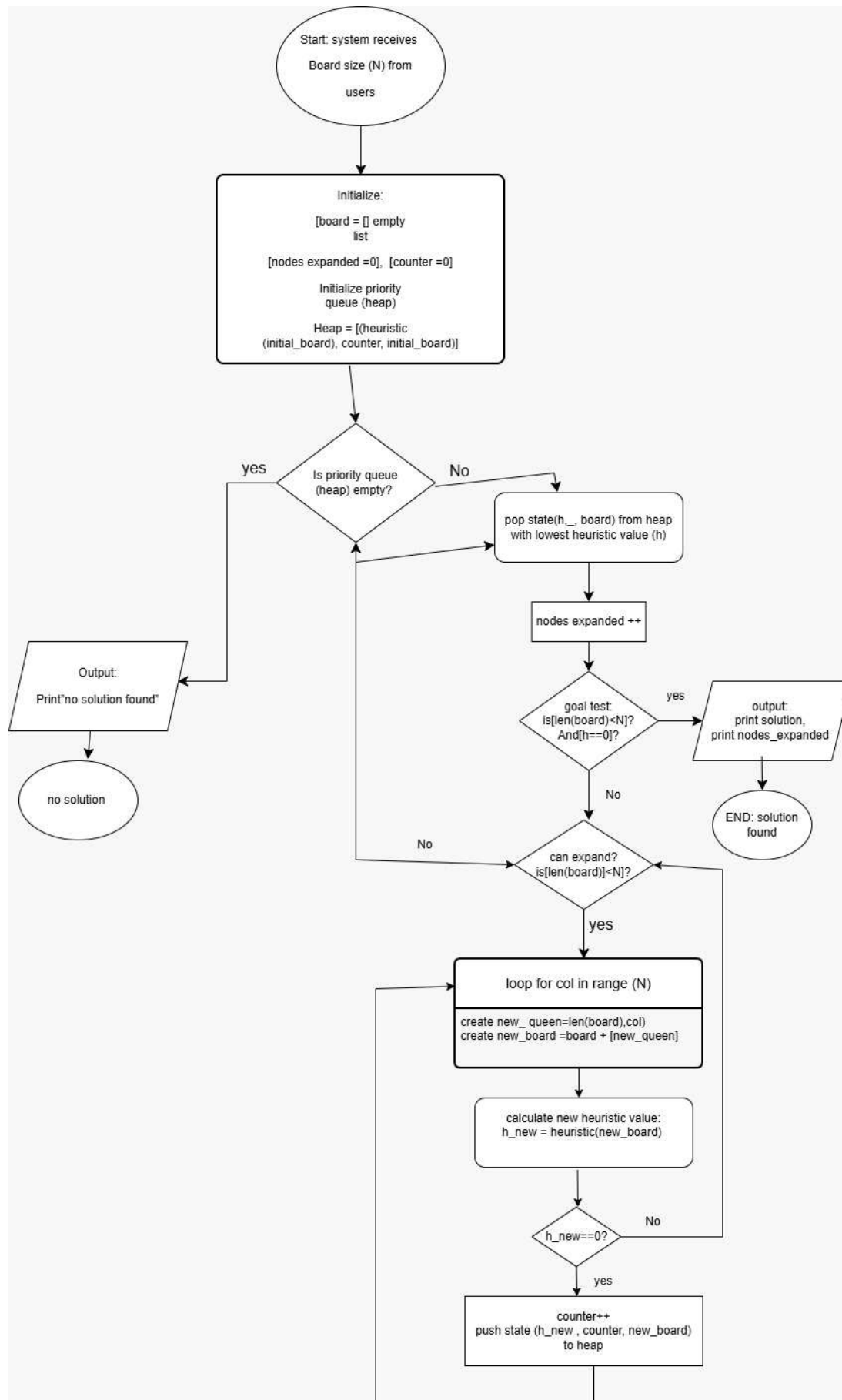


Flowchart for N-queens hill-climbing



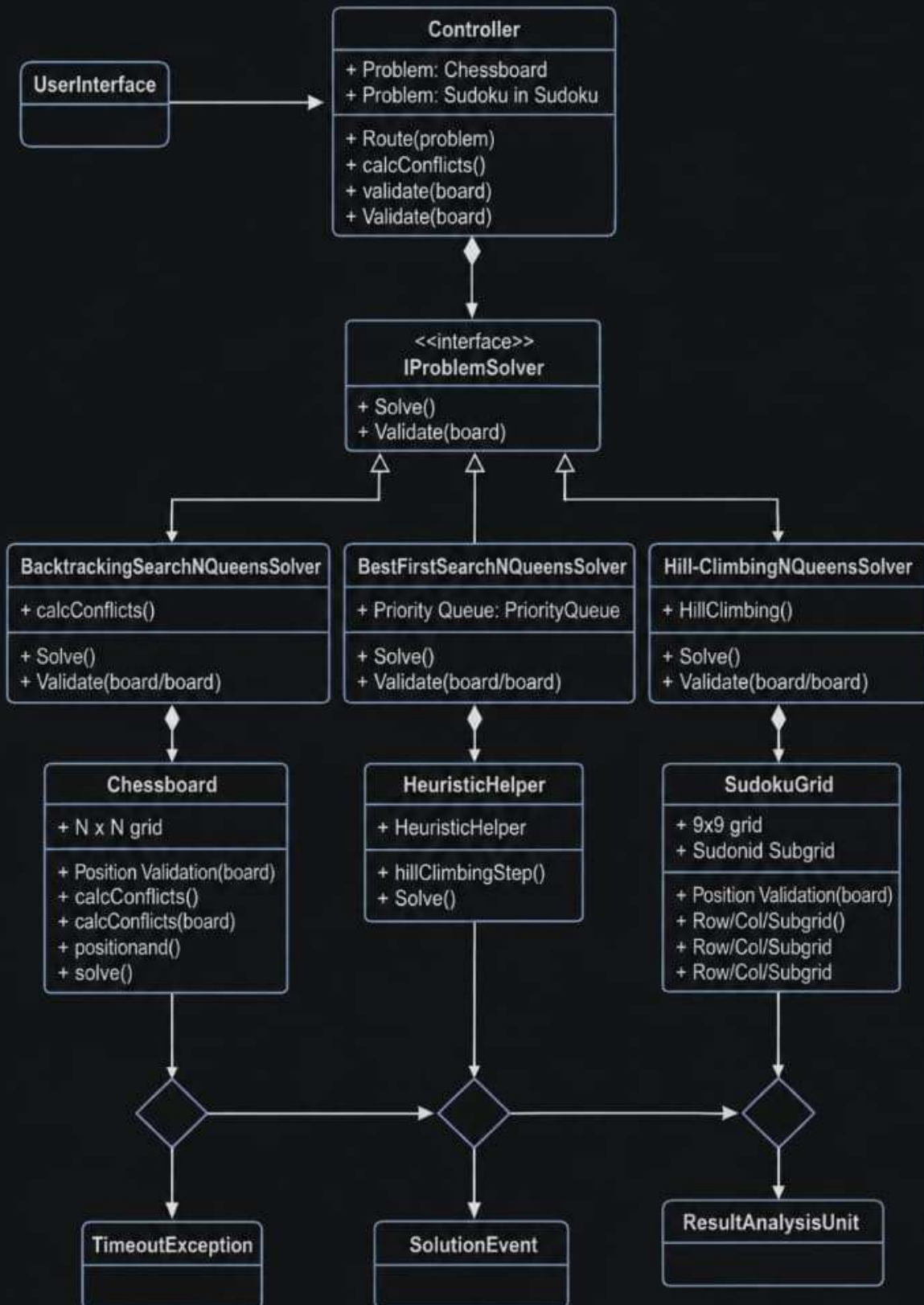
Best-First Search:





UML SYSTEM DIAGRAM

AI PROBLEM SOLVERS (N-QUEENS & SUDOKU)



6.7 Code Implementation

BackTracking:

```
27 n = int(input("n-queens : "))
28
29 board = [-1] * n
30
31 def isvalid(i,j):
32     for row in range(i):
33         if board[row] == j or abs(row-i) == abs(board[row]-j):
34             return False
35     return True
36
37 def dfs(i, current_values):
38     if i == n:
39         for row in current_values:
40             print(" ".join(row))
41         return True
42     for j in range(n):
43         if isvalid(i,j):
44             board[i] = j
45             if dfs(i+1, current_values + [ "."*j + "Q" + "." * (n-j-1)]):
46                 return True
47
48     return False
49
50 dfs(0, [])
```

Hill-Climbing:

```
1 import random
2 class NQueensSolver:
3     def __init__(self, n):
4         self.n = n
5     def calculate_conflicts(self, board):
6         conflicts = 0
7         for i in range(len(board)):
8             for j in range(i + 1, len(board)):
9                 if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
10                     conflicts += 1
11         return conflicts
12     def solve(self):
13         board = [random.randint(0, self.n - 1) for _ in range(self.n)]
14         history = []
15         steps = 0
```



```

16         while steps < 5000:
17             h = self.calculate_conflicts(board)
18             history.append(h)
19             if h == 0:
20                 return board, history, steps
21             best_board = list(board)
22             current_h = h
23             for col in range(self.n):
24                 original_row = board[col]
25                 for row in range(self.n):
26                     if row == original_row:
27                         continue
28                     board[col] = row
29                     new_h = self.calculate_conflicts(board)
30                     if new_h < current_h:
31                         current_h = new_h
32                         best_board = list(board)
33                 board[col] = original_row
34             if current_h >= h:
35                 board = [random.randint(0, self.n - 1) for _ in range(self.n)]
36             else:
37                 board = best_board
38             steps += 1
39         return None, history, steps
40     def print_board(board):
41         n = len(board)
42         for row in range(n):
43             line = ""
44             for col in range(n):
45                 if board[col] == row:
46                     line += " Q "
47                 else:
48                     line += " . "
49             print(line)

```

```

if __name__ == "__main__":
    try:
        n_input = input("Enter the number of queens (e.g., 8): ")
        n = int(n_input)
        print(f"\nSearching for a solution for {n}-Queens...")
        solver = NQueensSolver(n)
        solution, history, total_steps = solver.solve()
        if solution:
            print(f"--- Solution Found in {total_steps} iterations ---")
            print_board(solution)
            print("\nFinal State (Column indices):", solution)
            print("Conflict History (last 10 steps):", history[-10:])
        else:
            print("Could not find a solution within the iteration limit.")
    except ValueError:
        print("Please enter a valid integer.")

```

Best-First Search:

```
1  import heapq
2  def heuristic(queens):
3      conflicts = 0
4      for i in range(len(queens)):
5          for j in range(i + 1, len(queens)):
6              r1, c1 = queens[i]
7              r2, c2 = queens[j]
8              if r1 == r2 or c1 == c2 or abs(r1 - r2) == abs(c1 - c2):
9                  conflicts += 1
10     return conflicts
11 def best_first_search(n):
12     counter = 0
13     initial_board = []
14     heap = [(heuristic(initial_board), counter, initial_board)]
15     nodes_expanded = 0
16     while heap:
17         h, _, board = heapq.heappop(heap)
18         nodes_expanded += 1
19         if len(board) == n and h == 0:
20             print(f"Solution found after expanding {nodes_expanded} nodes.")
21             return board
22         if len(board) < n:
23             row = len(board)
24             for col in range(n):
25                 new_board = board + [(row, col)]
26                 counter += 1
27                 h_value = heuristic(new_board)
28                 heapq.heappush(heap, (h_value, counter, new_board))
29     print(f"No solution found after expanding {nodes_expanded} nodes.")
30     return None
31 solution = int(input("Enter the number of queens: "))
32 print(best_first_search(solution))
```

Genetic Algorithm:

```
1  import random
2  import heapq
3  # ===== CONFIG =====
4  MAX_N = 30
5  POP_SIZE = 80
6  MUTATION_RATE = 0.2
7  GENERATIONS = 1200
8  N = 8
```

```

9  # ===== FITNESS =====
10 def fitness(board):
11     conflicts = 0
12     cols = set()
13     diag1 = set()
14     diag2 = set()
15
16     for i in range(N):
17         c = board[i]
18
19         if c in cols:
20             conflicts += 1
21         else:
22             cols.add(c)
23
24         d1 = i - c
25         d2 = i + c
26
27         if d1 in diag1:
28             conflicts += 1
29         else:
30             diag1.add(d1)
31
32         if d2 in diag2:
33             conflicts += 1
34         else:
35             diag2.add(d2)
36
37     return -conflicts
38 # ===== POPULATION =====
39 def create_population():
40     return [[random.randint(0, N - 1) for _ in range(N)] for _ in range(POP_SIZE)]
41 # ===== SELECTION =====
42 def selection(pop):
43     return heapq.nlargest(30, pop, key=fitness)
44 # ===== CROSSOVER =====
45 def crossover(p1, p2):
46     point = random.randint(1, N - 2)
47     return p1[:point] + p2[point:]

```

```

48 # ===== MUTATION =====
49 def mutate(board):
50     board = board[:]
51     if random.random() < MUTATION_RATE:
52         idx = random.randint(0, N - 1)
53         board[idx] = random.randint(0, N - 1)
54     return board

```

```

55 # ===== GA =====
56 def genetic_algorithm():
57     pop = create_population()
58     for _ in range(GENERATIONS):
59         pop = selection(pop)
60         new_pop = pop[:]
61         while len(new_pop) < POP_SIZE:
62             p1, p2 = random.sample(pop, 2)
63             child = mutate(crossover(p1, p2))
64             new_pop.append(child)
65         pop = new_pop
66         best = max(pop, key=fitness)
67         if fitness(best) == 0:
68             return best
69     return max(pop, key=fitness)
70 # ===== PRINT BOARD =====
71 def print_board(board):
72     for i in range(N):
73         row = ["." for _ in range(N)]
74         row[board[i]] = "Q"
75         print(" ".join(row))
76 # ===== MAIN =====
77 def main():
78     global N
79     N = int(input("Enter N (e.g. 8): "))
80     if N > MAX_N:
81         N = MAX_N
82     solution = genetic_algorithm()
83     print("\nSolution:\n")
84     print_board(solution)
85 if __name__ == "__main__":
86     main()

```

6.7 The GUI Code:

```

1  import tkinter as tk
2  from tkinter import ttk
3  import time
4  import random
5  import heapq
6  class NQueensGUI:
7      def __init__(self):
8          self.root = tk.Tk()
9          self.root.title("N-Queens AI Solver")
10         self.root.geometry("520x620")
11         self.root.resizable(False, False)

```

```

12         self.solution = None
13         self.time_taken = 0
14         self.algorithm_name = ""
15         self.steps = 0
16         self.font_main = ("Times New Romain", 10)
17         self.font_title = ("Times New Romain", 14, "bold")
18         self.create_widgets()
19         # ===== UI =====
20         def create_widgets(self):
21             title = tk.Label(self.root, text="N-Queens Solver", font=self.font_title)
22             title.pack(pady=8)
23             frame = tk.Frame(self.root)
24             frame.pack(pady=5)
25             tk.Label(frame, text="N:", font=self.font_main).grid(row=0, column=0)
26             self.n_var = tk.IntVar(value=8)
27             tk.Entry(frame, textvariable=self.n_var, width=5).grid(row=0, column=1, padx=5)
28             tk.Label(frame, text="Algorithm:", font=self.font_main).grid(row=0, column=2)
29             self.algo_var = tk.StringVar(value="Backtracking (DFS)")
30             self.algo_combo = ttk.Combobox(
31                 frame,
32                 textvariable=self.algo_var,
33                 width=22,
34                 state="readonly",
35                 values=[
36                     "Backtracking (DFS)",
37                     "Hill Climbing",
38                     "Best First Search",
39                     "Genetic Algorithm"
40                 ]
41             )
42             self.algo_combo.grid(row=0, column=3, padx=5)
43             tk.Button(
44                 self.root,
45                 text="Solve",
46                 bg=█ "#826251",
47                 fg="white",
48                 font=self.font_main,
49                 command=self.solve
50             ).pack(pady=8)

```

```

51         self.info = tk.Label(self.root, text="", font=self.font_main)
52         self.info.pack()
53         # Chess board
54         self.canvas = tk.Canvas(self.root, width=400, height=400, bg="white")
55         self.canvas.pack(pady=10)
56         self.status = tk.Label(self.root, text="Ready", fg="gray", font=("Segoe UI", 9))
57         self.status.pack()

```

```

58 # ===== BOARD =====
59 def draw_board(self, board):
60     self.canvas.delete("all")
61     n = len(board)
62     size = 400 // n
63     light = "FFD1DC"
64     dark = "#826251"
65     for i in range(n):
66         for j in range(n):
67             x1, y1 = j * size, i * size
68             x2, y2 = x1 + size, y1 + size
69             color = light if (i + j) % 2 == 0 else dark
70             self.canvas.create_rectangle(x1, y1, x2, y2, fill=color, outline="black")
71             if board[i] == j:
72                 self.canvas.create_text(
73                     x1 + size // 2,
74                     y1 + size // 2,
75                     text="♟",
76                     font=("Times New Roman", size // 2),
77                     fill="black"
78                 )
79 # ===== BACKTRACKING =====
80 def backtracking(self, n):
81     board = [-1] * n
82     self.steps = 0
83     def safe(r, c):
84         for i in range(r):
85             if board[i] == c or abs(board[i] - c) == abs(i - r):
86                 return False
87         return True
88     def dfs(r):
89         self.steps += 1
90         if r == n:
91             return True
92         for c in range(n):
93             if safe(r, c):
94                 board[r] = c
95                 if dfs(r + 1):
96                     return True
97                 board[r] = -1
98         return False
99     start = time.perf_counter()
100     dfs(0)
101     self.time_taken = time.perf_counter() - start
102     return board
103 # ===== HILL CLIMBING =====
104 def hill_climbing(self, n):
105     self.steps = 0
106     def conflicts(b):
107         c = 0
108         for i in range(n):
109             for j in range(i + 1, n):
110                 if b[i] == b[j] or abs(b[i] - b[j]) == abs(i - j):

```

```

111         c += 1
112     return c
113     start = time.perf_counter()
114     board = [random.randint(0, n - 1) for _ in range(n)]
115     for _ in range(4000):
116         self.steps += 1
117         h = conflicts(board)
118         if h == 0:
119             break
120         best = board[:]
121         best_h = h
122         for col in range(n):
123             old = board[col]
124             for row in range(n):
125                 board[col] = row
126                 new_h = conflicts(board)
127                 if new_h < best_h:
128                     best_h = new_h
129                     best = board[:]
130             board[col] = old
131         if best_h >= h:
132             board = [random.randint(0, n - 1) for _ in range(n)]
133         else:
134             board = best
135     self.time_taken = time.perf_counter() - start
136     return board
137 # ===== BEST FIRST =====
138 def best_first(self, n):
139     self.steps = 0
140     def h(b):
141         c = 0
142         for i in range(len(b)):
143             for j in range(i + 1, len(b)):
144                 if b[i] == b[j] or abs(b[i] - b[j]) == abs(i - j):
145                     c += 1
146     return c

```

```

147     start = time.perf_counter()
148     heap = [(0, [])]
149     visited = set()
150     while heap:
151         self.steps += 1
152         _, board = heapq.heappop(heap)
153         if len(board) == n:
154             if h(board) == 0:
155                 self.time_taken = time.perf_counter() - start

```

```

156         return board
157     continue
158     for col in range(n):
159         new = board + [col]
160         t = tuple(new)
161         if t in visited:
162             continue
163         visited.add(t)
164         heapq.heappush(heap, (h(new), new))
165     self.time_taken = time.perf_counter() - start
166     return None
167 # ===== GENETIC =====
168 def genetic(self, n):
169     self.steps = 0
170     POP = 60
171     GEN = 300
172     MUT = 0.15
173     def fitness(b):
174         c = 0
175         for i in range(n):
176             for j in range(i + 1, n):
177                 if b[i] == b[j] or abs(b[i] - b[j]) == abs(i - j):
178                     c += 1
179         return -c
180     def cross(a, b):
181         p = random.randint(1, n - 2)
182         return a[:p] + b[p:]
183     start = time.perf_counter()
184     pop = [[random.randint(0, n - 1) for _ in range(n)] for _ in range(POP)]
185     for _ in range(GEN):
186         self.steps += 1
187         pop = sorted(pop, key=fitness, reverse=True)[:20]
188         new = pop[:]
189         while len(new) < POP:
190             a, b = random.sample(pop, 2)
191             child = cross(a, b)
192             if random.random() < MUT:
193                 i = random.randint(0, n - 1)
194                 child[i] = random.randint(0, n - 1)
195             new.append(child)
196         pop = new
197         best = max(pop, key=fitness)
198         if fitness(best) == 0:
199             self.time_taken = time.perf_counter() - start
200             return best
201     self.time_taken = time.perf_counter() - start
202     return max(pop, key=fitness)

```



```

203 # ===== SOLVE =====
204 def solve(self):
205     n = self.n_var.get()
206     algo = self.algo_var.get()
207     if n == 2 or n == 3:
208         self.solution = None
209         self.canvas.delete("all")
210         self.info.config(
211             text=f"No solution exists for N = {n}",
212             fg="red"
213         )
214         self.status.config(text="Done")
215         return
216     self.status.config(text="Solving...")
217     if algo == "Backtracking (DFS)":
218         self.solution = self.backtracking(n)
219         self.algorithm_name = "Backtracking"
220     elif algo == "Hill Climbing":
221         self.solution = self.hill_climbing(n)
222         self.algorithm_name = "Hill Climbing"
223     elif algo == "Best First Search":
224         self.solution = self.best_first(n)
225         self.algorithm_name = "Best First Search"
226     else:
227         self.solution = self.genetic(n)
228         self.algorithm_name = "Genetic"
229     if self.solution:
230         self.draw_board(self.solution)
231         self.info.config(
232             text=f"{self.algorithm_name} | Time: {self.time_taken:.6f}s | Steps: {self.steps}",
233             fg="green"
234         )
235     else:
236         self.info.config(text="No solution found", fg="red")
237     self.status.config(text="Done")
238 def run(self):
239     self.root.mainloop()
240 # ===== RUN =====
241 if __name__ == "__main__":
242     NQueensGUI().run()

```

6.8 Analysis of Results

The experimental results show that:

- ♠ Backtracking guarantees correct solutions but becomes inefficient for large values of N.
- ♠ Best-First Search improves performance using heuristics but depends on evaluation quality.
- ♠ Hill Climbing is very fast but may fail due to local optima.
- ♠ Genetic Algorithm provides a good balance between performance and solution quality, especially for large N.
- ♠ No. of Steps Taken by Backtracking is the same as the Best-First Search.
- ♠ Hill Climbing may get stuck in local optima.

7. Development Platform

- ♠ Programming Language: Python
- ♠ Libraries Used:
 - random
 - heapq
- ♠ Execution Environment:
 - Command Line Interface (CLI)
- ♠ Hardware Requirements:
 - Any standard computer capable of running Python

8. Conclusion

This project demonstrates different approaches to solving the N-Queens problem using:

- ♠ Exact algorithms (Backtracking)
- ♠ Heuristic methods (Best-First and Hill Climbing)
- ♠ Evolutionary algorithms (Genetic Algorithm)

It highlights the trade-offs between:

- ♠ Accuracy
- ♠ Performance
- ♠ Scalability

The project also provides an interactive environment where users can explore and compare different artificial intelligence technology.

Link for files:

https://drive.google.com/drive/folders/1Elpw__Wgac-ZZtX7thMlBwFOCoK1bi6c?usp=sharing